NutriPredict

E4 : CI/CD, DevOps et Monitoring

Candidat : Valentin PERIES

Certification : RNCP37827 - Developpeur en Intelligence Artificielle

Projet : NutriPredict - Prediction d'impact nutritionnel par IA

Date : Juin 2026

C14 C15 C16 C17

E4 - CI/CD, DevOps et Monitoring

1. Strategie d'integration continue (C14)

L'integration continue (CI) du projet NutriPredict est geree par **GitHub Actions**. Le workflow `backend-ci.yml` est declenche automatiquement a chaque push sur les branches `main` et `bloc-2-ai-research`, ainsi que sur les pull requests.

1.1 Architecture du pipeline CI

Le pipeline est compose de trois jobs executees en parallele ou en sequence selon leurs dependances :

Job	Dependance	Actions	Duree
Lint and Type Check	Aucune	Flake8 (erreurs critiques) + MyPy (typage)	~56s
Backend Tests	Lint	Pytest avec PostgreSQL 16 en service	~52s
ML Model Validation	Aucune	Validation des performances du modele	~33s

1.2 Job "Lint and Type Check"

Ce job execute deux analyses statiques :

- **Flake8** : analyse syntaxique en deux passes. La premiere cible les erreurs critiques (`E9, F63, F7, F82`) avec echec du pipeline. La seconde mesure la complexite cyclomatique (seuil = 10) sans bloquer.
- **MyPy** : verification des types Python. Execute en mode tolerant (`--ignore-missing-imports`) pour les bibliotheques tierces non typees.

1.3 Job "Backend Tests"

Ce job lance les tests Pytest dans un environnement identique a la production :

- Un service PostgreSQL 16 est demarre via un container Docker dans GitHub Actions.
- Les variables d'environnement (cle de chiffrement, URL de la base, cle API) sont injectees via la section `env` du workflow.
- Le `PYTHONPATH` est configure pour resoudre les imports du backend.

```
# Extrait du workflow CI - Job Backend Tests
services:
  db:
    image: postgres:16
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: test_db
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 5s
      --health-timeout 3s
      --health-retries 5
```

1.4 Job "ML Model Validation"

Ce job est independant et verifie que le modele maintient ses performances :

- Charge le dataset et le modele serialise.
- Execute les predictions sur le jeu de test.
- Verifie que le R2 > 0.95 et la MAE < 5g (seuils d'acceptabilite).
- Echec du pipeline si les performances se degradent.

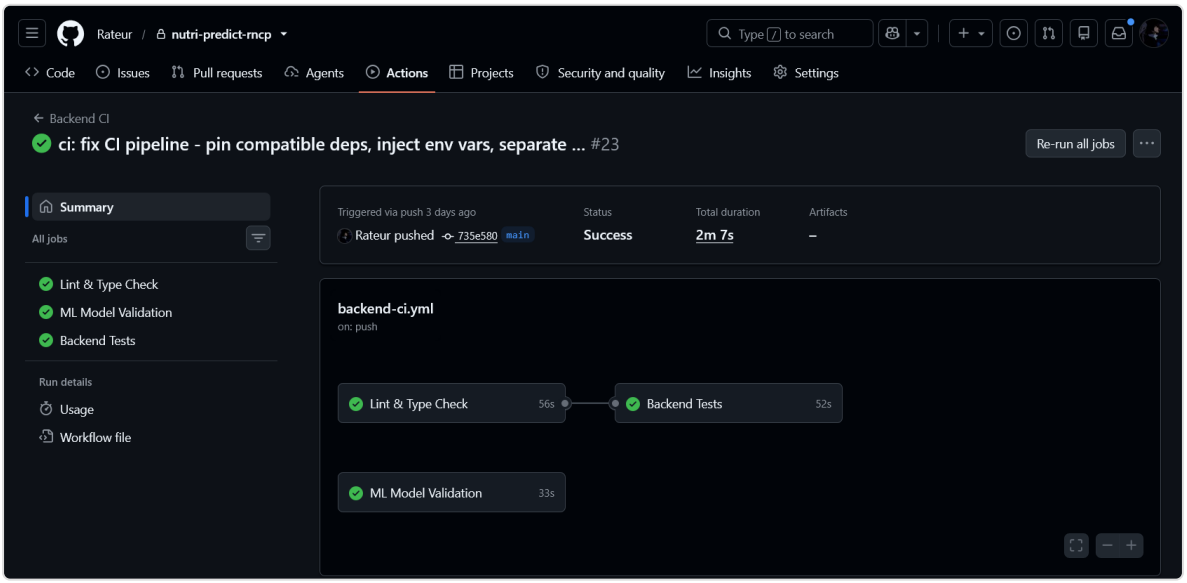


Figure 14 - Pipeline CI : 3 jobs reussis (Lint, Tests, ML Validation)

2. Deploiement continu (C15)

Le pipeline de deploiement continu (CD) est declenche par la creation d'un **tag Git** au format semantique (ex: `v1.0.0`). Il construit et publie l'image Docker sur le registre GitHub Container Registry (GHCR).

2.1 Workflow CD

Le fichier `cd-deployment.yml` execute les etapes suivantes :

1. **Checkout** : recuperation du code source au commit du tag.
2. **Authentification GHCR** : connexion au registre via le token `GITHUB_TOKEN` (secret automatique GitHub Actions).
3. **Build Docker** : construction de l'image depuis `backend/Dockerfile`.
4. **Push** : publication de l'image avec les tags extraits automatiquement (version semantique + latest).

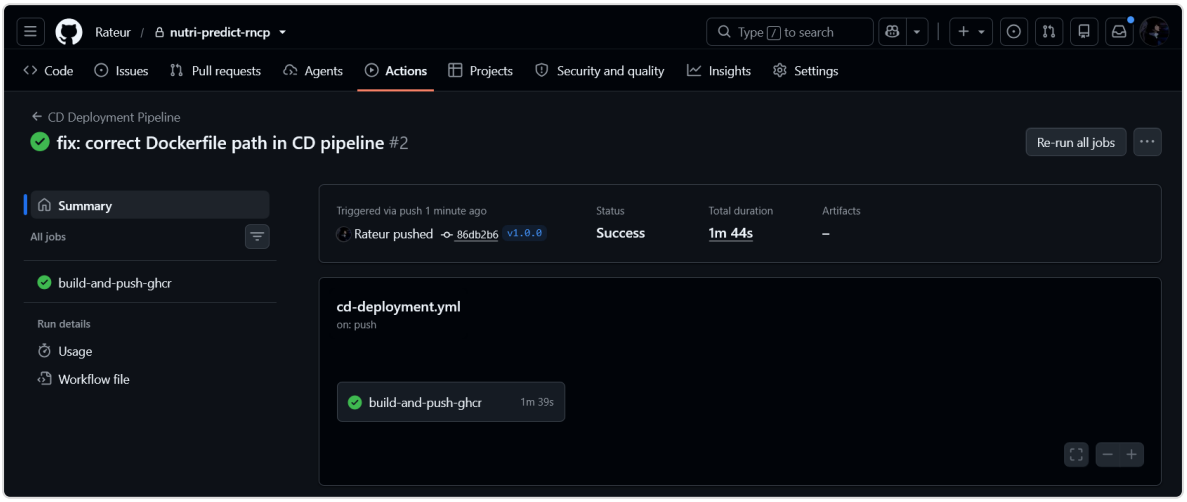


Figure 15 - Pipeline CD : build and push GHCR reussi (1m 44s)

3. Gestion de version et branching (C16)

3.1 Strategie de branches

Le projet utilise un modele de branching simplifie :

- **main** : branche de production, protegee par le pipeline CI.
- **bloc-2-ai-research** : branche de recherche pour les experimentations ML, fusionnee dans main via merge commit.
- **Tags semantiques** : `v1.0.0` pour marquer les releases et declencher le CD.

```
* 86db2b6 (HEAD -> main, tag: v1.0.0, origin/main) fix: correct Dockerfile path in CD pipeline
* 5847431 docs: rewrite README to match actual project structure and fix all references
* 735e580 ci: fix CI pipeline - pin compatible deps, inject env vars, separate lint/test/ML jobs
* 0bac18c fix: resolve flake8 F821 undefined name using TYPE_CHECKING
* f419c40 ci: optimize pipeline to run faster by skipping docker build and using pip cache
* fdd62c6 docs: restructure README with RNCP competencies matrix
* e43b407 chore: remove temporary scripts after setup
*   df7225e Merge branch 'bloc-2-ai-research' into main: production release
|\
| * 06fecdd (bloc-2-ai-research) docs: write comprehensive README for RNCP certification
| * 67d849a (origin/bloc-2-ai-research) chore: add .env.example
| * ef13de8 docs: write RAPPORT_E4.md for certification
| * 0c1dac4 docs: write RAPPORT_E3.md for certification
| * 0c04479 docs: document BENCHMARK_IA.md
| * 9442387 fix: patch issue in 02_model_training.ipynb
| * 55edfc4 fix: resolve bug in VEILLE_TECHNOLOGIQUE.md
| * df2ad40 test: add tests in test_products.py
| * 910d173 test: add tests in generate_test_split.py
| * 2727c79 feat: implement ml_service.py
| * bdc6426 docs: write RAPPORT_E2.md for certification
| * ab03da5 chore: update config 01_eda_predictive_modeling.ipynb
```

Figure 16 - Historique Git : branches, merges et tags

3.2 Gestion de projet agile

Le suivi des taches est realise via **GitHub Projects** avec un tableau Kanban a trois colonnes : Backlog, En cours, Termine.

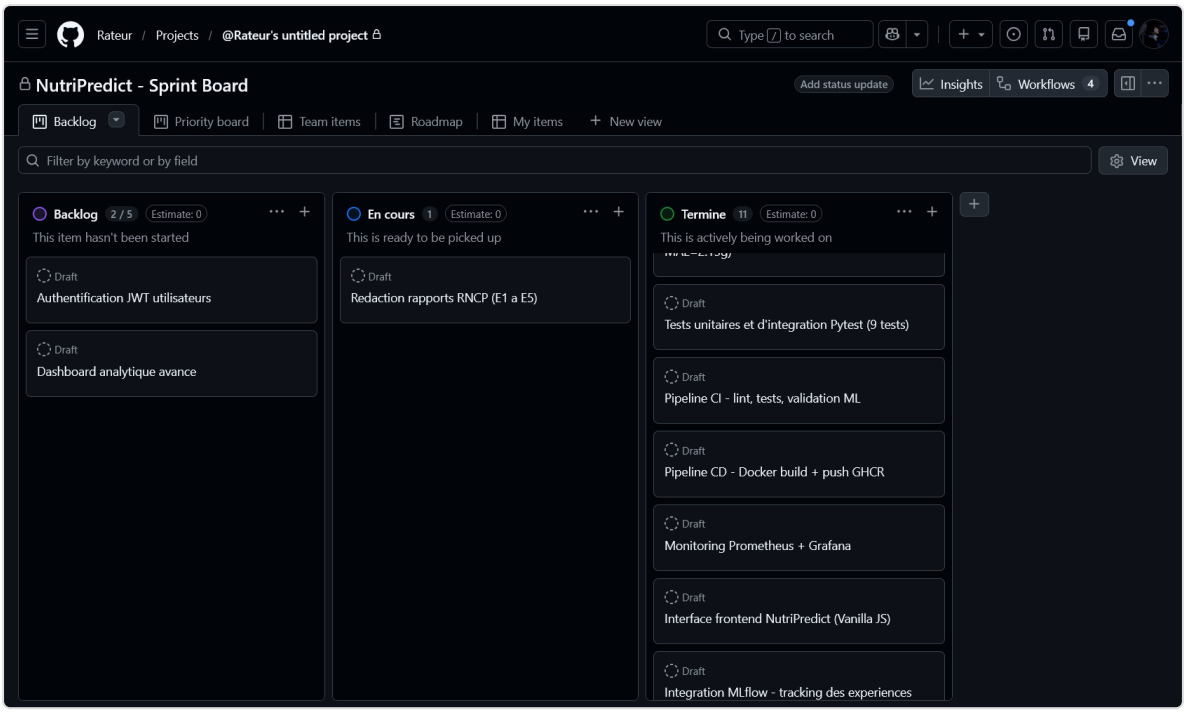


Figure 17 - GitHub Projects : tableau Kanban du sprint

4. Conteneurisation Docker (C16)

4.1 Dockerfile

Le Dockerfile utilise une image de base legere (`python:3.12-slim`) pour minimiser la taille de l'image finale :

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends curl
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PYTHONPATH=/app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.2 Docker Compose

L'orchestration locale des 4 services est geree par Docker Compose. Les points notables :

- Le service `api` depend de `db` via `condition: service_healthy` , ce qui empeche le demarrage de l'API tant que PostgreSQL n'est pas pret.
- Les secrets sont injectes via `env_file: .env` (fichier non versionne).
- Prometheus scrape l'API toutes les 15 secondes sur l'endpoint `/metrics` .

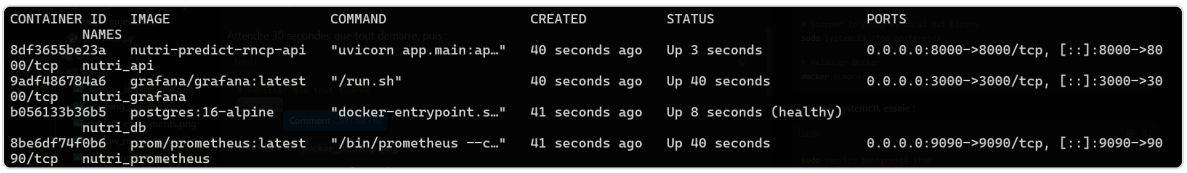


Figure 18 - Docker Compose : 4 conteneurs en fonctionnement

5. Monitoring et observabilite (C17)

5.1 Metriques Prometheus

L'API expose des metriques au format OpenMetrics sur l'endpoint `/metrics` , generees automatiquement par `prometheus-fastapi-instrumentator` . Les metriques incluent :

- **http_request_duration_seconds** : histogramme de la latence par endpoint.
- **http_requests_total** : compteur total de requetes par methode/endpoint/status.
- **ml_weight_delta_grams** : histogramme personnalise des predictions ML.

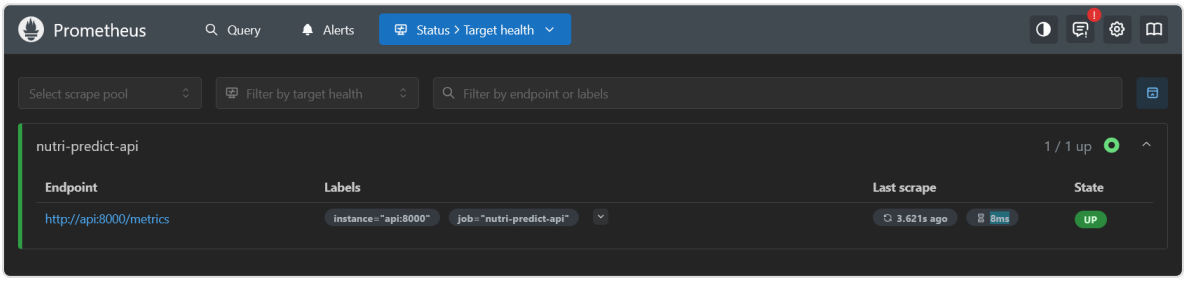


Figure 19 - Prometheus : target nutri-predict-api en etat UP

5.2 Dashboards Grafana

Un dashboard pre-configure est provisionne automatiquement au demarrage de Grafana via des fichiers YAML de provisioning. Il affiche :

- **Requests per second (RPS)** : debit de requetes par endpoint, permettant de detecter les pics de charge.
- **Latency P95** : le 95e percentile de la latence, indicateur de la qualite de service percue par les utilisateurs.

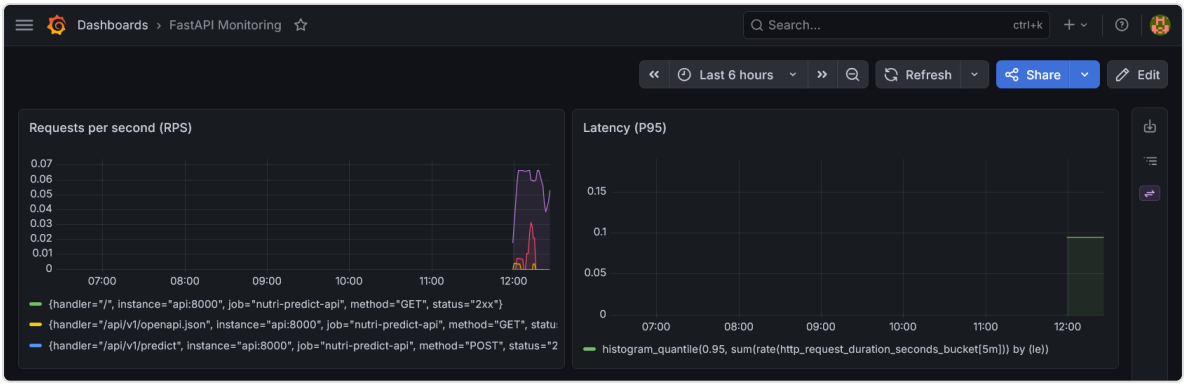


Figure 20 - Grafana : dashboard FastAPI Monitoring (RPS et Latency P95)

5.3 Logs structures

Les logs de l'application sont emis au format JSON via `structlog`, ce qui facilite leur ingestion par des outils de centralisation (ELK, Loki). Chaque log inclut :

- Timestamp ISO 8601
- Niveau de log (info, error, warning)
- Nom du module source
- Donnees contextuelles (BMR, portion, duree d'inference)

```
{"event": "predict_weight_success", "predicted_delta": -26.5,
 "duration_seconds": 0.008, "level": "info",
 "timestamp": "2026-06-19T10:19:47Z"}
```